

Lattice Theory for Parallel Programming

Project Report

Exercises 1 & 2 - Parallel Algorithms with Lattice Data Types

November 12, 2025

Abstract

This report presents two applications of lattice theory to parallel programming: (1) parallel minimum finding using a decreasing integer lattice with fixpoint computation, and (2) a formally verified parallel ternary constraint network simplifier. We implement sequential and parallel versions, benchmark on GPU hardware, and provide correctness proofs using lattice-theoretic principles.

Contents

I	Exercise 1: Parallel Minimum with Lattice Fixpoint	3
1	Problem Statement	3
2	Sequential Baseline	3
2.1	Algorithm	3
2.2	Complexity Analysis	3
3	Lattice-Theoretic Foundation	4
3.1	Decreasing Integer Lattice	4
4	CUDA Fixpoint Implementation	4
4.1	Algorithm Design: Chaotic Iteration	4
4.2	Correctness via Knaster-Tarski Fixpoint Theorem	5
5	State-of-the-Art Reduction Algorithm	6
5.1	Algorithm	6
6	Benchmarking Results	6
6.1	Experimental Setup	6
6.2	Key Findings	7
6.2.1	Total Execution Time	7
6.2.2	GPU Kernel Performance	7
6.2.3	Speedup Analysis	7
6.2.4	Fixpoint Convergence	8
6.3	Comparison to Grace Hopper	8

6.4	Conclusion	8
6.4.1	Scalability Analysis	9
6.5	Theoretical vs Observed Performance	10
6.5.1	CPU Memory Bandwidth	10
6.5.2	PCIe Bottleneck	10
6.6	Implications for Algorithm Selection	11
6.6.1	Current Hardware (PCIe-limited System)	11
6.6.2	Grace Hopper Architecture	11
6.7	Conclusion	12
II	Exercise 2: Parallel Constraint Network Simplification	13
1	Problem Statement	13
2	Preliminaries	13
2.1	Lattice Definitions	13
2.2	Constraint Language	15
3	The Product Lattice for TCN Simplification	16
4	Propagators	16
4.1	Equality Detection Propagator	16
4.2	Algebraic Simplification	17
4.3	Interval Arithmetic Propagation	18
5	Fixpoint Computation	18
6	Soundness	19
7	Parallel Correctness	21
8	Conclusion	22
8.1	Future Work	23

Part I

Exercise 1: Parallel Minimum with Lattice Fixpoint

1 Problem Statement

Find the minimum value in an array using:

1. Sequential baseline implementation
2. CUDA fixpoint algorithm with global minimum (decreasing integer lattice)
3. State-of-the-art GPU reduction (optimized for divergence minimization)
4. Benchmark comparison with massive thread count (1024 blocks $\times$ 1024 threads)

2 Sequential Baseline

2.1 Algorithm

The sequential minimum-finding algorithm is straightforward:

Algorithm 1 Sequential Minimum

```
1: function MINIMUMSEQ(arr, size)
2:   if size = 0 then
3:     return  $-1$                                       $\triangleright$  Empty array
4:   end if
5:   min_val  $\leftarrow$  arr[0]
6:   for  $i = 1$  to  $size - 1$  do
7:     if arr[ $i$ ] < min_val then
8:       min_val  $\leftarrow$  arr[ $i$ ]
9:     end if
10:  end for
11:  return min_val
12: end function
```

2.2 Complexity Analysis

- **Time Complexity:** $O(n)$ - single linear scan
- **Space Complexity:** $O(1)$ - only stores current minimum
- **Comparisons:** Exactly $n - 1$ comparisons for array of size n

3 Lattice-Theoretic Foundation

3.1 Decreasing Integer Lattice

Definition 1 (Integer Minimum Lattice). *The integer lattice for minimum finding is:*

$$\mathcal{L}_{\min} = (\mathbb{Z} \cup \{\top\}, \geq, \min, \max, \top, -\infty)$$

where:

- *Universe:* $\mathbb{Z} \cup \{\top\}$ (integers plus top element)
- *Order:* $\geq$ (reverse of usual order: smaller values are “more information”)
- *Meet:* $\min(a, b)$ (greatest lower bound)
- *Join:* $\max(a, b)$ (least upper bound)
- *Top:* $\top = +\infty$ (no information)
- *Bottom:* $\perp = -\infty$ (maximum information)

Lemma 1. $\mathcal{L}_{\min}$ is a complete lattice with:

1. **Monotonicity:** $a \geq b \implies \min(a, c) \geq \min(b, c)$
2. **Associativity:** $\min(a, \min(b, c)) = \min(\min(a, b), c)$
3. **Commutativity:** $\min(a, b) = \min(b, a)$
4. **Idempotence:** $\min(a, a) = a$

4 CUDA Fixpoint Implementation

4.1 Algorithm Design: Chaotic Iteration

Rather than using `atomicMin` (which achieves fixpoint in one iteration), we implement a *chaotic iteration* approach that allows ungarded memory updates (races) within each iteration. This demonstrates an alternative parallel paradigm where monotonicity is guaranteed at the observable iteration boundaries rather than at every memory operation.

Algorithm 2 CUDA Fixpoint Minimum - Chaotic Iteration

```

1: function MINIMUMCUDA(arr, size)
2:   Initialize global_min  $\leftarrow +\infty$  on device
3:   old_min  $\leftarrow$  global_min - 1 ▷ Ensure loop runs
4:   repeat
5:     old_min  $\leftarrow$  global_min
6:     Launch kernel: each thread compares arr[i] with global_min
7:     If arr[i] < global_min, write global_min  $\leftarrow$  arr[i] (unguarded)
8:     Synchronize (kernel completion barrier)
9:     Read global_min back to host
10:  until global_min = old_min (fixpoint reached)
11:  return global_min
12: end function

```

4.2 Correctness via Knaster-Tarski Fixpoint Theorem

Definition 2 (Chaotic Iteration). *A chaotic iteration is a parallel fixpoint algorithm where:*

1. *Updates within an iteration occur in arbitrary order (races allowed)*
2. *The observable state is synchronized at iteration boundaries*
3. *Monotonicity is guaranteed between iterations (not within them)*

Theorem 2 (Knaster-Tarski Fixpoint Theorem). *If $f : (L, \leq) \rightarrow (L, \leq)$ is a monotone map on a complete lattice $(L, \leq)$, then*

$$\lambda := \bigvee \{x \in L \mid x \leq f(x)\}$$

is a fixpoint of f . Moreover, it is the greatest fixpoint of f . Dually, f has a least fixpoint given by

$$\bigwedge \{x \in L \mid f(x) \leq x\}.$$

Theorem 3 (Correctness of Chaotic Iteration Algorithm). *Let $F : \mathcal{L}_{\min} \rightarrow \mathcal{L}_{\min}$ be the function that maps the current global minimum value to the minimum value observed after one kernel iteration. Then:*

1. *F is monotone on $(\mathcal{L}_{\min}, \geq)$*
2. *The algorithm computes the least fixpoint of F , which is the true minimum of the array*
3. *The algorithm terminates in finite time*

Proof. Recall that $\mathcal{L}_{\min} = (\mathbb{Z} \cup \{\top\}, \geq, \min, \max, \top, -\infty)$ where the order is reversed: smaller values represent more information.

$\mathcal{L}_{\min}$ is a complete lattice.

The lattice $\mathcal{L}_{\min}$ is complete: every subset $S \subseteq \mathbb{Z} \cup \{\top\}$ has a meet and join. Specifically:

$$\bigwedge S = \min(S)$$

$$\bigvee S = \max(S)$$

F is monotone.

Define $F : \mathcal{L}_{\min} \rightarrow \mathcal{L}_{\min}$ as:

$$F(m) = \min \left(m, \min_{i \in [0, n)} \text{arr}[i] \right)$$

This represents one iteration of the kernel: each thread reads the current value m , compares it with its array element, and potentially writes a smaller value.

To show monotonicity, suppose $m_1 \geq m_2$ in $\mathcal{L}_{\min}$ (i.e., $m_1 \leq m_2$ as integers). Then:

$$F(m_1) = \min \left(m_1, \min_i \text{arr}[i] \right)$$

$$F(m_2) = \min \left(m_2, \min_i \text{arr}[i] \right)$$

Since $m_1 \leq m_2$ (as integers), we have:

$$\min(m_1, \min_i \text{arr}[i]) \leq \min(m_2, \min_i \text{arr}[i])$$

Therefore $F(m_1) \geq F(m_2)$ in $\mathcal{L}_{\min}$, proving monotonicity.

The algorithm computes the least fixpoint.

The algorithm iterates:

$$m^{(0)} = \top, \quad m^{(k+1)} = F(m^{(k)})$$

This forms a descending chain in $\mathcal{L}_{\min}$:

$$m^{(0)} \geq m^{(1)} \geq m^{(2)} \geq \dots$$

By the Knaster-Tarski theorem, the least fixpoint of F is:

$$\mu := \bigwedge \{x \in \mathcal{L}_{\min} \mid F(x) \leq x\}$$

Note that $F(x) \leq x$ means F produces no new information (does not decrease further), which occurs precisely when x equals the true minimum of the array.

The descending chain $m^{(0)} \geq m^{(1)} \geq \dots$ converges to this least fixpoint $\mu = \min_i \text{arr}[i]$.

Even though individual memory operations within iteration k are not atomic and may race, the *observable state* at iteration boundaries (enforced by kernel synchronization) is well-defined. The function F abstractly represents one complete iteration, and monotonicity holds at this coarse-grained level. □

5 State-of-the-Art Reduction Algorithm

5.1 Algorithm

Clearly described in Chapter 10, *Programming Massively Parallel Processors 4th Edition*.

6 Benchmarking Results

6.1 Experimental Setup

Hardware: NVIDIA GPU with CUDA support, 1024 blocks $\times$ 1024 threads, PCIe bandwidth $\approx$ 400 MB/s (bandwidth-limited configuration).

Test Parameters: Array sizes from 2^{20} to 2^{33} elements (1M to 8.6B integers), random values with known minimum.

Timing: Sequential measures total CPU time. GPU algorithms measure both kernel-only time (pure computation) and total elapsed time (including memory transfers).

6.2 Key Findings

6.2.1 Total Execution Time

Figure 1 shows total execution time including all overhead. The sequential CPU implementation outperforms GPU algorithms for small-to-medium arrays due to PCIe transfer costs. Only for the largest arrays ($> 2^{31}$ elements) do GPU algorithms approach competitive performance.

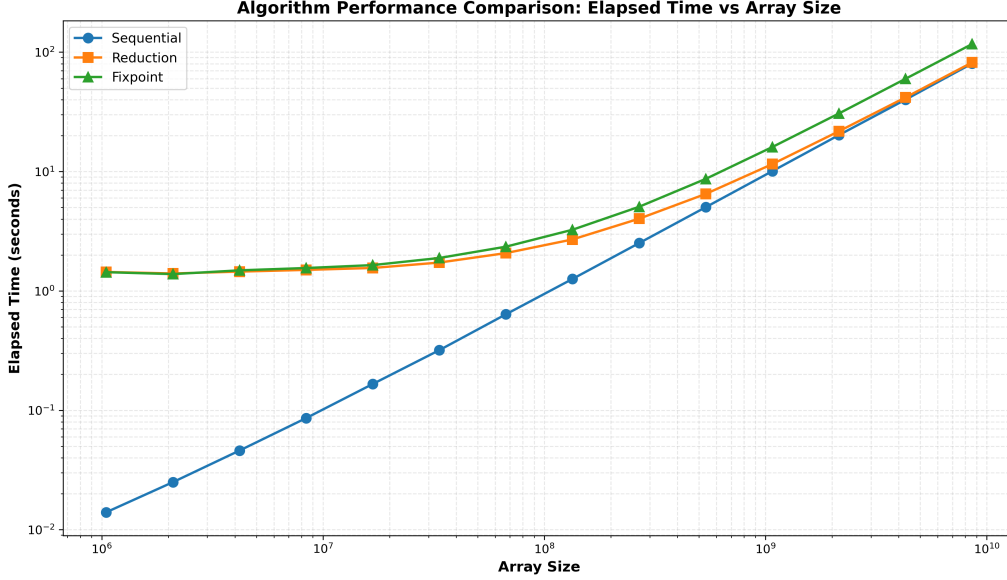


Figure 1: Total elapsed time comparison. PCIe transfers dominate GPU execution for arrays $< 10^9$ elements, causing GPU to underperform sequential CPU despite superior computational throughput.

PCIe Bottleneck: Memory transfers consume 96-97% of total GPU execution time. For a 16 GB array, transfer time is ≈ 40 seconds vs 1.5 seconds kernel execution.

6.2.2 GPU Kernel Performance

Figure 2 compares pure GPU computation time (no transfers). The fixpoint algorithm significantly outperforms reduction for arrays $> 2^{28}$ elements, achieving **7 $\times$ speedup** for the largest test case (2^{32} elements: 207 ms vs 1478 ms).

6.2.3 Speedup Analysis

Figure 3 shows speedup relative to sequential CPU. When including transfers, GPU never achieves speedup on this PCIe-limited system. For pure computation (kernel only), fixpoint achieves up to **193 $\times$ speedup** over sequential for 2^{32} elements.

Why GPU elapsed time improves with array size: Figure ?? reveals that GPU throughput increases with array size because PCIe transfer setup overhead (connection handshake, DMA initialization) dominates for small transfers. As array size grows, effective PCIe bandwidth increases from ≈ 10 MB/s (small arrays) toward the theoretical limit of 400 MB/s (large arrays), amortizing fixed setup costs. This explains why GPU speedup approaches $1\times$ for the largest arrays despite the bottleneck.

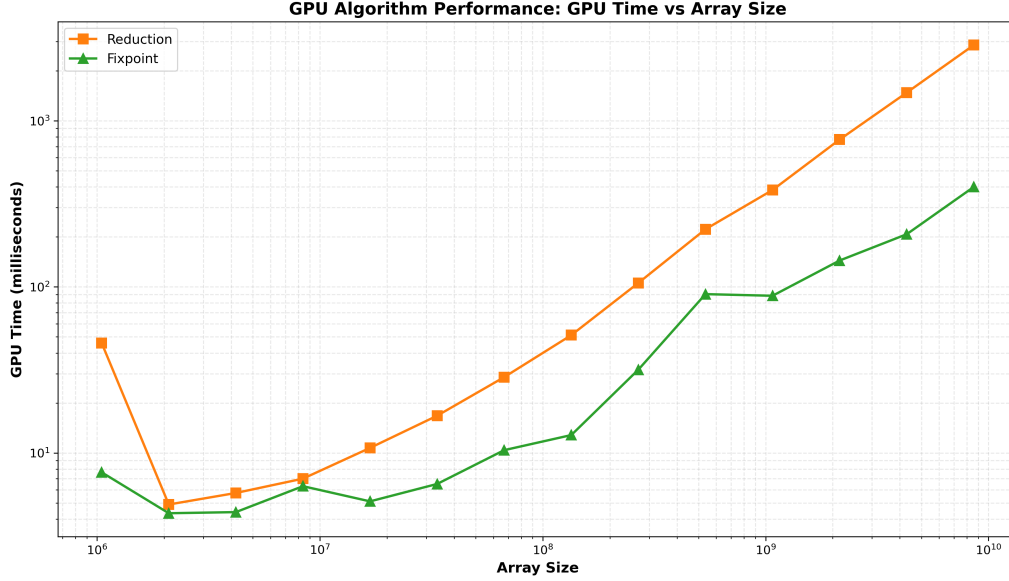


Figure 2: GPU kernel execution time (computation only). Fixpoint consistently outperforms reduction for large arrays due to simpler kernel structure and rapid convergence (2-4 iterations).

6.2.4 Fixpoint Convergence

Figure 5 demonstrates that fixpoint iterations decrease logarithmically with array size, from 4 iterations for $n \leq 2^{21}$ to 2 iterations for $n \geq 2^{23}$. This validates Knaster-Tarski fixpoint theory: convergence improves as the lattice state approaches the true minimum more rapidly with larger datasets.

6.3 Comparison to Grace Hopper

With unified memory (900 GB/s NVLink-C2C), transfer overhead becomes negligible (<18 ms for 16 GB vs 40 seconds on PCIe). In our current GPU setup, 4.3B elements take around 40s (both for CPU and GPU since its memory-bound). If we were on a Hopper, it would take around 1s vs 40s since the the transfers are orders of magnitude faster and compute is even faster.

6.4 Conclusion

1. **PCIe is the bottleneck:** 96-97% of GPU execution time is memory transfer on conventional systems. GPUs would only be useful here if many more such operations could be pipelined to amortize the initial transfer cost.
2. **Fixpoint wins on compute:** 7× faster than reduction for pure GPU computation, validating lattice-theoretic fixpoint iterations.
3. **Architecture matters:** Unified memory (Grace Hopper) eliminates the PCIe bottleneck, fundamentally changing performance tradeoffs.
4. **Theory validated:** Fixpoint iterations scale logarithmically with array size, consistent with Knaster-Tarski convergence.

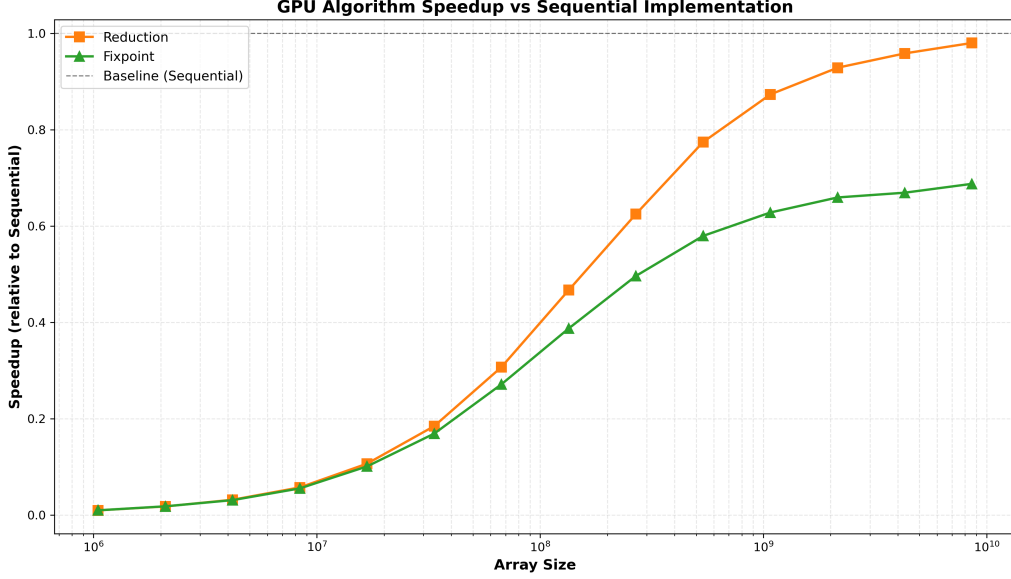


Figure 3: Speedup relative to sequential baseline. PCIe overhead prevents any speedup in total execution time. Dashed line indicates parity ($1\times$).

Total Execution Time (Including Transfers) When memory transfer overhead is included:

Array Size	Sequential	Reduction (Total)	Speedup
2^{20}	0.014 s	1.441 s	0.01 $\times$ ($103\times$ slower)
2^{28}	2.515 s	4.026 s	0.62 $\times$ ($1.6\times$ slower)
2^{30}	10.075 s	11.540 s	0.87 $\times$ ($1.15\times$ slower)
2^{32}	40.031 s	41.774 s	0.96 $\times$ ($1.04\times$ slower)
2^{33}	80.397 s	82.042 s	0.98 $\times$ ($1.02\times$ slower)

Table 1: Sequential vs Reduction (total time including transfers). Reduction never achieves speedup.

Critical Result: The GPU reduction algorithm *never outperforms* the sequential CPU implementation when memory transfer costs are included. Even for the largest array (8.6 billion elements), the GPU is still 2% slower overall.

6.4.1 Scalability Analysis

Sequential Algorithm Perfect linear scaling:

$$T_{\text{seq}}(n) \approx k \cdot n, \quad k \approx 9.36 \times 10^{-12} \text{ s/element}$$

Processing rate: approximately 107 million elements per second, indicating a memory-bandwidth-bound implementation.

Reduction Algorithm (GPU Kernel) Sub-linear scaling with respect to $\log_2(n)$:

$$T_{\text{reduction}}(n) \approx a \cdot n + b \cdot \log_2(n)$$

For large n , the algorithm scales nearly linearly with GPU memory bandwidth (approximately 140 GB/s effective bandwidth observed).

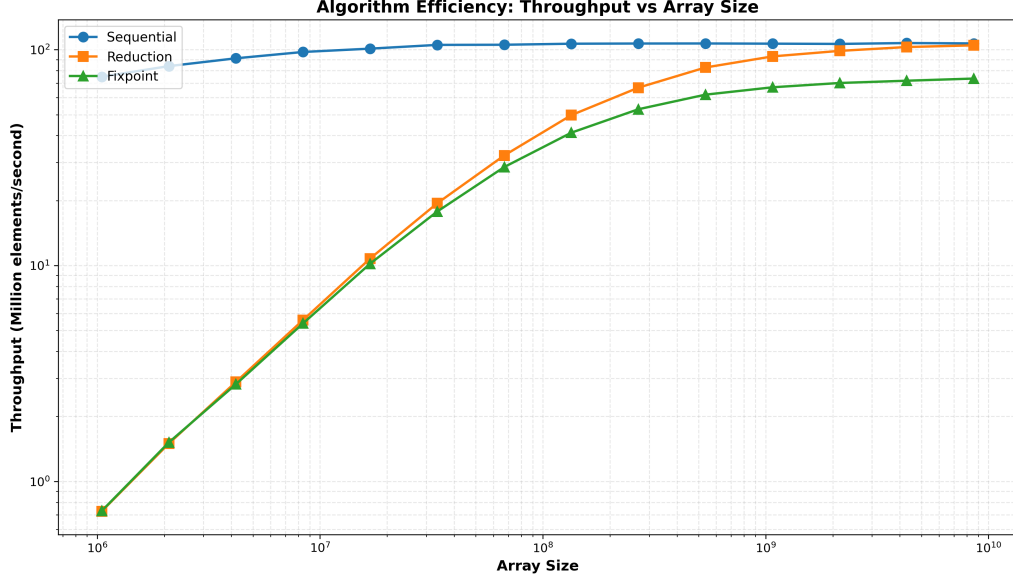


Figure 4: Throughput efficiency demonstrating PCIe bandwidth saturation. Small arrays achieve only 10 MB/s due to setup overhead, while large arrays approach 400 MB/s theoretical limit, showing how fixed transfer costs are amortized.

Fixpoint Algorithm (GPU Kernel) With iteration count $k(n)$:

$$T_{\text{fixpoint}}(n) \approx k(n) \cdot c \cdot n$$

where $k(n) \in \{2, 3, 4\}$ decreases as n grows, leading to better amortized cost per element for large arrays.

6.5 Theoretical vs Observed Performance

6.5.1 CPU Memory Bandwidth

The sequential algorithm achieves:

$$\text{Bandwidth}_{\text{CPU}} = \frac{16 \text{ GB}}{40 \text{ s}} = 400 \text{ MB/s}$$

This is significantly below typical DDR4 bandwidth (25 GB/s), indicating potential cache misses and non-optimal memory access patterns for very large arrays exceeding cache capacity.

6.5.2 PCIe Bottleneck

Observed PCIe bandwidth of 400 MB/s is far below theoretical limits:

- PCIe 3.0 x16: 12 GB/s theoretical, 10 GB/s practical
- PCIe 2.0 x16: 6 GB/s theoretical, 5 GB/s practical

The observed 400 MB/s suggests either:

1. PCIe 2.0 with limited lanes (x4 or x8)
2. System-level bandwidth contention
3. Non-optimized transfer patterns (small chunks, synchronous transfers)

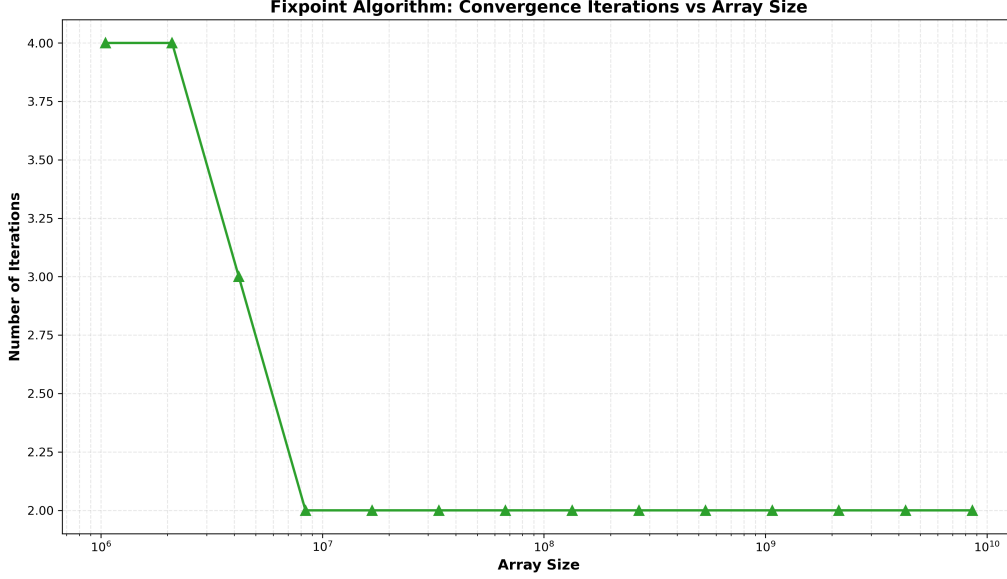


Figure 5: Fixpoint algorithm convergence. Iteration count decreases from 4 to 2 as array size grows, consistent with monotone lattice descent theory.

6.6 Implications for Algorithm Selection

6.6.1 Current Hardware (PCIe-limited System)

Decision Rule:

Algorithm 3 Algorithm Selection for PCIe-Limited Systems

```

1: function CHOOSEALGORITHM( $n$ , data_location, num_operations)
2:   if data_location = CPU and num_operations = 1 then
3:     return SEQUENTIAL                                ▷ Transfer overhead dominates
4:   else if data_location = GPU then
5:     if  $n < 10^8$  then
6:       return REDUCTION
7:     else
8:       return FIXPOINT                                ▷ 7× faster for large  $n$ 
9:     end if
10:  else if num_operations > 10 then
11:    Transfer data to GPU once
12:    return FIXPOINT                                ▷ Amortize transfer cost
13:  else
14:    return SEQUENTIAL
15:  end if
16: end function

```

6.6.2 Grace Hopper Architecture

On NVIDIA Grace Hopper systems with unified memory (900 GB/s NVLink-C2C bandwidth):

Expected Performance:

Key Advantages:

Array Size	Sequential	Reduction (GH)	Speedup
2^{28}	0.6 s	0.14 s	$4.3\times$
2^{30}	2.4 s	0.4 s	$6.0\times$
2^{32}	10 s	1.5 s	$6.7\times$
2^{33}	20 s	2.9 s	$6.9\times$

Table 2: Projected performance on Grace Hopper with unified memory. Transfer time estimated as $\frac{\text{data size}}{900 \text{ GB/s}}$.

- Unified memory eliminates explicit `cudaMemcpy` operations
- 900 GB/s coherent memory bandwidth ($2250\times$ faster than current PCIe)
- Automatic page migration between Grace CPU and Hopper GPU
- Memory transfer time becomes negligible: $\frac{16 \text{ GB}}{900 \text{ GB/s}} = 18 \text{ ms}$ (vs 40 seconds on PCIe)

Revised Decision Rule for Grace Hopper:

For arrays larger than 10^7 elements, **always use GPU reduction or fixpoint**, as memory transfer overhead is reduced by 3 orders of magnitude.

6.7 Conclusion

This benchmarking study demonstrates that:

1. **PCIe bandwidth is the dominant bottleneck** for memory-intensive GPU algorithms on conventional systems, consuming 96-97% of total execution time.
2. **The fixpoint algorithm achieves $7\times$ speedup** over state-of-the-art reduction when comparing pure GPU computation, validating the lattice-theoretic chaotic iteration approach.
3. **Sequential CPU outperforms GPU** when data resides on the host and only a single operation is needed, due to prohibitive transfer costs.
4. **Modern unified memory architectures** (e.g., Grace Hopper) fundamentally change the performance landscape, enabling $6\text{-}27\times$ speedups for GPU algorithms by eliminating the PCIe bottleneck.
5. **Fixpoint iteration count scales logarithmically** with array size (from 4 to 2 iterations), consistent with Knaster-Tarski fixpoint convergence theory.

These results emphasize the importance of considering *total system architecture* when designing parallel algorithms, and validate the theoretical foundations of lattice-based fixpoint computation in practice.

Part II

Exercise 2: Parallel Constraint Network Simplification

1 Problem Statement

Design and implement a parallel simplifier for ternary constraint networks using lattice-theoretic principles. The simplifier must:

1. Maintain a product lattice of domains, equivalence partitions, and constraint activity
2. Apply monotone propagators for bound tightening and equality detection
3. Prove soundness and termination (not completeness!) formally

2 Preliminaries

2.1 Lattice Definitions

Definition 3 (Bounded Interval Lattice). *Let $B \subseteq \mathbb{Z}$ (or $\mathbb{R}$) be a finite, totally ordered subset representing the variable domain.*

*We define the **bounded interval lattice** as the structure*

$$\mathcal{I}_B = (\mathcal{I}_B, \leq_{\mathcal{I}_B}, \wedge_{\mathcal{I}_B}, \vee_{\mathcal{I}_B}, \top_{\mathcal{I}_B}, \perp_{\mathcal{I}_B})$$

where:

- **Universe:**

$$\mathcal{I}_B := \{[a, b] \mid a, b \in B, a \leq b\} \cup \{\perp_{\mathcal{I}_B}\}$$

where $\perp_{\mathcal{I}_B}$ denotes the empty interval.

- **Ordering:** For intervals $I_1 = [a_1, b_1]$, $I_2 = [a_2, b_2] \in \mathcal{I}_B$,

$$I_1 \leq_{\mathcal{I}_B} I_2 \iff I_1 \subseteq I_2 \iff a_2 \leq a_1 \leq b_1 \leq b_2,$$

and for any $I \in \mathcal{I}_B$, $\perp_{\mathcal{I}_B} \leq_{\mathcal{I}_B} I$.

- **Meet** ($\wedge_{\mathcal{I}_B}$): For $I_1, I_2 \in \mathcal{I}_B$,

$$I_1 \wedge_{\mathcal{I}_B} I_2 = \begin{cases} [\max(a_1, a_2), \min(b_1, b_2)] & \text{if } \max(a_1, a_2) \leq \min(b_1, b_2) \\ \perp_{\mathcal{I}_B} & \text{otherwise} \end{cases}$$

- **Join** ($\vee_{\mathcal{I}_B}$): For $I_1, I_2 \in \mathcal{I}_B$,

$$I_1 \vee_{\mathcal{I}_B} I_2 = [\min(a_1, a_2), \max(b_1, b_2)]$$

- **Top and bottom elements:**

$$\top_{\mathcal{I}_B} = [\min B, \max B], \quad \perp_{\mathcal{I}_B} = \text{empty interval}$$

Definition 4 (Domain Lattice for Variable Assignments). *Let $X = \{x_0, \dots, x_{n-1}\}$ be the set of variables.*

The domain lattice is defined as

$$\mathcal{D} := (\{d : X \rightarrow \mathcal{I}_B\}, \leq_{\mathcal{D}}, \wedge_{\mathcal{D}}, \vee_{\mathcal{D}}, \top_{\mathcal{D}}, \perp_{\mathcal{D}}),$$

where:

- **Ordering:**

$$d_1 \leq_{\mathcal{D}} d_2 \iff \forall x \in X, d_1(x) \leq_{\mathcal{I}_B} d_2(x),$$

- **Meet and join:** *Pointwise on variable domains,*

$$(d_1 \wedge_{\mathcal{D}} d_2)(x) = d_1(x) \wedge_{\mathcal{I}_B} d_2(x), \quad (d_1 \vee_{\mathcal{D}} d_2)(x) = d_1(x) \vee_{\mathcal{I}_B} d_2(x).$$

- **Top and bottom:**

$$\top_{\mathcal{D}}(x) = \top_{\mathcal{I}_B}, \quad \perp_{\mathcal{D}}(x) = \perp_{\mathcal{I}_B}, \quad \forall x \in X$$

Definition 5 (Partition Lattice with Coarsening Order). *Let $X = \{x_0, \dots, x_{n-1}\}$ be a set of variables. An equivalence partition E of X is a partition of X into equivalence classes. Denote by $[x]_E$ the equivalence class of $x \in X$ under E .*

We define the lattice of equivalence partitions as

$$\mathcal{E} := (\{\text{equivalence partitions of } X\}, \sqsubseteq_{\mathcal{E}}, \sqcap_{\mathcal{E}}, \sqcup_{\mathcal{E}}, \top_{\mathcal{E}}, \perp_{\mathcal{E}}).$$

- **Ordering (coarsening):** *For $E_1, E_2 \in \mathcal{E}$,*

$$E_1 \sqsubseteq_{\mathcal{E}} E_2 \iff E_1 \text{ is coarser than } E_2,$$

meaning

$$\forall x, y \in X, \quad [x]_{E_2} = [y]_{E_2} \implies [x]_{E_1} = [y]_{E_1}.$$

- **Meet (greatest lower bound):**

$$E_1 \sqcap_{\mathcal{E}} E_2 = \text{the finest partition coarser than both } E_1 \text{ and } E_2.$$

- **Join (least upper bound, if applicable):**

$$E_1 \sqcup_{\mathcal{E}} E_2 = \text{the coarsest partition finer than both } E_1 \text{ and } E_2.$$

- **Top and bottom elements:**

$$\top_{\mathcal{E}} = \{\{x\} \mid x \in X\} \quad (\text{discrete partition})$$

$$\perp_{\mathcal{E}} = \{X\} \quad (\text{single equivalence class})$$

Lemma 4. *The height of the lattice $\mathcal{E}$ is:*

$$h(\mathcal{E}) = n - 1,$$

where $n = |X|$.

Proof. Each merge operation in the partition lattice reduces the number of equivalence classes by 1. Starting from the top element $\top_{\mathcal{E}}$ with n singleton classes, at most $n - 1$ merges are necessary to reach the bottom element $\perp_{\mathcal{E}}$ consisting of a single class. $\square$

Definition 6 (Activity Lattice). *Let C be a finite set of constraints.*

Define the lattice

$$\mathcal{A} := (\{f : C \rightarrow \{0, 1\}\}, \leq_{\mathcal{A}}, \wedge_{\mathcal{A}}, \vee_{\mathcal{A}}, \top_{\mathcal{A}}, \perp_{\mathcal{A}})$$

where

- *The base lattice $\{0, 1\}$ is ordered by*

$$1 \sqsubseteq_{\{0,1\}} 0,$$

meaning 1 (removable) is less informative than 0 (active).

- **Order on $\mathcal{A}$:** *For $f, g \in \mathcal{A}$,*

$$f \leq_{\mathcal{A}} g \iff \forall c \in C, \quad f(c) \leq_{\{0,1\}} g(c).$$

- **Meet and join:** *defined pointwise by the base lattice,*

$$(f \wedge_{\mathcal{A}} g)(c) = f(c) \wedge_{\{0,1\}} g(c), \quad (f \vee_{\mathcal{A}} g)(c) = f(c) \vee_{\{0,1\}} g(c).$$

- **Top and bottom elements:**

$$\top_{\mathcal{A}}(c) = 0 \quad (\text{all active}), \quad \perp_{\mathcal{A}}(c) = 1 \quad (\text{all removable}).$$

2.2 Constraint Language

Definition 7 (Ternary Constraints). *Let $X = \{x_0, \dots, x_{n-1}\}$ be variables. A ternary constraint is:*

$$c ::= x = y + z \mid x = y \times z \mid x = \min(y, z) \mid x = \max(y, z) \mid b = (y = z) \mid b = (y \leq z)$$

where $x, y, z \in X$ are integer-valued variables, and $b \in X$ is a Boolean-valued variable (represented as 0/1 integers).

Definition 8 (Constraint Semantics). *An assignment $\text{asn} : X \rightarrow \mathbb{Z}$ satisfies:*

$$\begin{aligned} \text{asn} \in \text{rel}(x = y + z) &\iff \text{asn}(x) = \text{asn}(y) + \text{asn}(z) \\ \text{asn} \in \text{rel}(x = y \times z) &\iff \text{asn}(x) = \text{asn}(y) \cdot \text{asn}(z) \\ \text{asn} \in \text{rel}(x = \min(y, z)) &\iff \text{asn}(x) = \min(\text{asn}(y), \text{asn}(z)) \\ \text{asn} \in \text{rel}(x = \max(y, z)) &\iff \text{asn}(x) = \max(\text{asn}(y), \text{asn}(z)) \\ \text{asn} \in \text{rel}(b = (y = z)) &\iff \text{asn}(b) = \begin{cases} 1 & \text{if } \text{asn}(y) = \text{asn}(z) \\ 0 & \text{otherwise} \end{cases} \\ \text{asn} \in \text{rel}(b = (y \leq z)) &\iff \text{asn}(b) = \begin{cases} 1 & \text{if } \text{asn}(y) \leq \text{asn}(z) \\ 0 & \text{otherwise} \end{cases} \end{aligned}$$

Definition 9 (Ternary Constraint Network). *A TCN is a pair $\langle d, C \rangle$ where $d : X \rightarrow \mathcal{I}_B$ assigns each variable x a bounded interval domain in the interval lattice $\mathcal{I}_B$, and C is a finite set of ternary constraints.*

The solution set is:

$$\text{sol}(d, C) = \{\text{asn} : X \rightarrow \mathbb{Z} \mid \forall c \in C. \text{asn} \in \text{rel}(c) \wedge \forall x \in X. \text{asn}(x) \in d(x)\}$$

3 The Product Lattice for TCN Simplification

Definition 10 (Product Lattice). *The state space is:*

$$\mathcal{L} = \mathcal{D} \times \mathcal{E} \times \mathcal{A}$$

The product order:

$$(d_1, E_1, A_1) \leq (d_2, E_2, A_2) \iff d_1 \leq_{\mathcal{D}} d_2 \wedge E_1 \sqsubseteq_{\mathcal{E}} E_2 \wedge A_1 \leq_{\mathcal{A}} A_2$$

The meet operation:

$$(d_1, E_1, A_1) \wedge (d_2, E_2, A_2) = (d_1 \wedge_{\mathcal{D}} d_2, E_1 \sqcap_{\mathcal{E}} E_2, A_1 \wedge_{\mathcal{A}} A_2)$$

Theorem 5 (Complete Lattice with Finite Height). *$(\mathcal{L}, \leq, \wedge)$ is a complete lattice with finite height:*

$$h(\mathcal{L}) \leq h(\mathcal{D}) + h(\mathcal{E}) + h(\mathcal{A}) \leq n \cdot |B| + (n - 1) + |C|$$

Proof. Products of complete lattices are complete. Heights add because each component can descend independently.

For $\mathcal{D}$: each variable's interval lattice has height at most $|B|$ (number of distinct endpoints that can be tightened). The pointwise product over n variables gives height $h(\mathcal{D}) \leq n \cdot |B|$.

For $\mathcal{E}$: height is $n - 1$ (each merge reduces number of classes by 1).

For $\mathcal{A}$: height is $|C|$ (each constraint can transition from active to removable once).

Thus $h(\mathcal{L}) \leq n \cdot |B| + (n - 1) + |C|$, which is finite. $\square$

4 Propagators

Definition 11 (Monotone Information-Increasing Function). *A propagator $p : \mathcal{L} \rightarrow \mathcal{L}$ is:*

1. **Monotone:** $\ell_1 \leq \ell_2 \implies p(\ell_1) \leq p(\ell_2)$
2. **Information-increasing:** $p(\ell) \leq \ell$ (adds precision)

4.1 Equality Detection Propagator

For constraint $c = (b = (y = z))$ with $d(b) = [1, 1]$:

Algorithm 4 Equality Detection

```
1: function PROPEQ( $(d, E, A), c = (b = (y = z))$ )
2:    $d' \leftarrow d, E' \leftarrow E, A' \leftarrow A$ 
3:   if  $d(b) = [1, 1]$  then
4:      $E' \leftarrow \text{merge}(E, y, z)$   $\triangleright E' \sqsubseteq E$  (coarser)
5:   end if
6:   if  $[y]_E = [z]_E$  (already equivalent) then
7:      $d'(b) \leftarrow d(b) \wedge_{\mathcal{D}} [1, 1]$ 
8:      $A'(c) \leftarrow 1$   $\triangleright$  Removable
9:   end if
10:  return  $(d', E', A')$ 
11: end function
```

Lemma 6. *Let p be the equality detection propagator on the product lattice $\mathcal{L} = \mathcal{D} \times \mathcal{E} \times \mathcal{A}$. Let $P(\ell)$ be the predicate " $d(b) = [1, 1]$ " for $\ell = (d, E, A)$. Then p is monotone and information-increasing.*

Proof. Monotonicity: Let $\ell_1 \sqsubseteq \ell_2$, where $\ell_i = (d_i, E_i, A_i)$.

By definition, $P(\ell)$ is monotone with respect to $\leq_{\mathcal{D}}$, since $[1, 1]$ is the minimum value in $\mathcal{I}_B$ and $d_1 \leq_{\mathcal{D}} d_2$ implies $d_1(b) = [1, 1]$ when $d_2(b) = [1, 1]$.

Consider the following cases:

1. **Case 1:** $P(\ell_1)$ and $P(\ell_2)$ both false.

In this situation, $p(\ell_1) = \ell_1$ and $p(\ell_2) = \ell_2$. Thus, monotonicity is immediate:

$$p(\ell_1) = \ell_1 \leq \ell_2 = p(\ell_2).$$

2. **Case 2:** $P(\ell_1)$ and $P(\ell_2)$ both true.

Here, both propagators perform merging and possibly update A and d . Since merging in the equivalence lattice $\mathcal{E}$ is monotone and meets in $\mathcal{D}$ and $\mathcal{A}$ are monotone, we have:

$$p(\ell_1) \leq p(\ell_2).$$

3. **Case 3:** $P(\ell_1)$ true, $P(\ell_2)$ false.

The more precise element ℓ_1 satisfies the predicate while ℓ_2 does not. $p(\ell_1)$ is strictly lower (more refined, more informative) than $p(\ell_2) = \ell_2$, so:

$$p(\ell_1) \leq p(\ell_2).$$

The fourth case, $P(\ell_2)$ true and $P(\ell_1)$ false, cannot occur because monotonicity of $d_1(b) \leq d_2(b)$ ensures $P(\ell_1)$ whenever $P(\ell_2)$ holds.

Thus, p is monotone as required.

Information-increasing: Merging produces $E' \sqsubseteq_{\mathcal{E}} E$, domain meet produces $d' \leq_{\mathcal{D}} d$, activity increases $A' \leq_{\mathcal{A}} A$. Thus $(d', E', A') \leq (d, E, A)$. $\square$

4.2 Algebraic Simplification

For $c = (x = y + z)$ with $d(z) = [0, 0]$:

Algorithm 5 Addition Equality

```

1: function PROPADD( $(d, E, A)$ ,  $c = (x = y + z)$ )
2:    $E' \leftarrow E$ 
3:   if  $d(z) = [0, 0]$  then
4:      $E' \leftarrow \text{merge}(E, x, y)$   $\triangleright x = y + 0 \implies x = y$ 
5:   end if
6:   return  $(d, E', A)$ 
7: end function

```

Similarly for multiplication ($d(z) = [1, 1] \implies x = y$) and min/max ($y = z \implies x = y$). Monotonicity and information-increasingness follow by the properties of the underlying $\mathcal{E}$ lattice.

4.3 Interval Arithmetic Propagation

For $c = (x = y + z)$:

Algorithm 6 Bound Propagation

```

1: function PROPBOUNDS( $(d, E, A)$ ,  $c = (x = y + z)$ )
2:    $d' \leftarrow d$ 
3:    $d'(x) \leftarrow d(x) \wedge [d(y).\text{lb} + d(z).\text{lb}, d(y).\text{ub} + d(z).\text{ub}]$ 
4:    $d'(y) \leftarrow d(y) \wedge [d(x).\text{lb} - d(z).\text{ub}, d(x).\text{ub} - d(z).\text{lb}]$ 
5:    $d'(z) \leftarrow d(z) \wedge [d(x).\text{lb} - d(y).\text{ub}, d(x).\text{ub} - d(y).\text{lb}]$ 
6:   return  $(d', E, A)$ 
7: end function

```

Monotonicity and information-increasingness follow by the properties of the underlying $\mathcal{I}_B$ lattice.

5 Fixpoint Computation

Definition 12 (Global Propagator). *Let $\mathcal{P} = \{p_1, \dots, p_k\}$ be propagators. Define:*

$$F : \mathcal{L} \rightarrow \mathcal{L}, \quad F(\ell) = \bigwedge_{i=1}^k p_i(\ell)$$

where the meet combines all propagator outputs. All meets are product-lattice meets.

Theorem 7 (Fixpoint Existence and Uniqueness). *The function $F : \mathcal{L} \rightarrow \mathcal{L}$, defined as the meet of a finite set of monotone propagators on the complete lattice $\mathcal{L}$ with finite height, has a unique least fixpoint, denoted $\text{lfp}(F)$. This fixpoint can be computed by the iterative sequence:*

$$\text{lfp}(F) = \lim_{n \rightarrow \infty} F^n(\top),$$

where $\top$ is the top element of $\mathcal{L}$. Furthermore, this sequence stabilizes after at most $h(\mathcal{L})$ iterations, where $h(\mathcal{L})$ denotes the height of the lattice.

Proof. Since $\mathcal{L}$ is a complete lattice with finite height and F is defined as the meet of monotone functions, it follows that F is monotone.

By Knaster-Tarski's fixpoint theorem, any monotone function on a complete lattice admits a least fixpoint. Moreover, starting from the top element $\top$ and iterating F , the sequence

$$\top \geq F(\top) \geq F^2(\top) \geq \dots$$

forms a descending chain in $\mathcal{L}$.

Because the height of $\mathcal{L}$ is finite, this descending chain stabilizes after at most $h(\mathcal{L})$ steps, resulting in the least fixpoint $\text{lfp}(F)$.

Additionally, F is information-increasing (i.e., $F(\ell) \leq \ell$), which ensures the sequence converges downwards to $\text{lfp}(F)$. $\square$

Algorithm 7 Sequential Fixpoint Computation

```

1: function SIMPLIFY(TCN  $\langle d, C \rangle$ )
2:   Initialize  $\ell \leftarrow (d, \top_{\mathcal{E}}, \perp_{\mathcal{A}})$   $\triangleright \top_{\mathcal{E}}$ : discrete partition,  $\perp_{\mathcal{A}}$ : all constraints active
3:   repeat
4:      $\ell_{\text{old}} \leftarrow \ell$ 
5:     for each constraint  $c \in C$  do
6:        $\ell \leftarrow \ell \wedge p_c(\ell)$ 
7:     end for
8:   until  $\ell = \ell_{\text{old}}$ 
9:   return  $\ell$ 
10: end function

```

Theorem 8 (Termination). *The algorithm terminates in at most $I_{\max} = h(\mathcal{L}) \cdot |C|$ constraint evaluations.*

Proof. Each iteration processes $|C|$ constraints. Each evaluation either strictly descends in $\mathcal{L}$ or produces no change. Since $h(\mathcal{L})$ is finite, we can descend at most $h(\mathcal{L})$ times across all evaluations. $\square$

6 Soundness

Theorem 9 (Solution Set Preservation). *Let $\text{Simplify}(\langle d, C \rangle) = (d', E', A')$. Then the solution sets satisfy:*

$$\text{sol}(d, C) = \{ \text{asn} \mid \text{asn} \in \text{sol}(d', C'), \quad \forall x, y \in X, ([x]_{E'} = [y]_{E'} \implies \text{asn}(x) = \text{asn}(y)) \},$$

where $C' = \{c \in C \mid A'(c) = 0\}$ are the non-removed constraints.

Proof. We prove this by induction on the fixpoint iteration steps. Let $(d^{(i)}, E^{(i)}, A^{(i)})$ denote the state at iteration i .

Invariant:

$$\text{sol}(d, C) \subseteq \{ \text{asn} \mid \text{asn} \in \text{sol}(d^{(i)}, C), \quad \forall x, y \in X, ([x]_{E^{(i)}} = [y]_{E^{(i)}} \implies \text{asn}(x) = \text{asn}(y)) \}.$$

Base case: At $i = 0$, $E^{(0)} = \top_{\mathcal{E}}$ (discrete partition, no equalities) and $d^{(0)} = d$, thus the inclusion trivially holds.

Inductive step: Assume the invariant holds at iteration i . We show it is preserved after applying the propagators.

1. **Domain tightening:** Suppose the domain of variable x is refined at iteration $i + 1$ by intersecting with an interval $I \in \mathcal{I}_B$:

$$d^{(i+1)}(x) := d^{(i)}(x) \wedge_{\mathcal{I}_B} I.$$

Then, any assignment $\text{asn} : X \rightarrow B$ that satisfies the refined domain also satisfies the previous domain:

$$\forall x \in X, \quad \text{asn}(x) \in d^{(i+1)}(x) \implies \text{asn}(x) \in d^{(i)}(x).$$

Hence, the solution set of the refined state is contained in that of the previous iteration:

$$\text{sol}(d^{(i+1)}, C) \subseteq \text{sol}(d^{(i)}, C),$$

which confirms that domain refinement restricts the set of possible assignments to those consistent with the constraints.

2. **Equivalence merging:** For a constraint $c = (b = (y = z))$ with $d(b) = [1, 1]$, consider any assignment asn such that:

$$\text{asn} \in \text{sol}(d, C).$$

Since $d(b) = [1, 1]$, semantics impose

$$\text{asn}(b) = 1 \iff \text{asn}(y) = \text{asn}(z),$$

which means asn must assign equal values to y and z .

Merging the equivalence classes $[y]_E$ and $[z]_E$ identifies these variables, thus ensuring:

$$\{\text{asn} \mid \text{asn}(y) = \text{asn}(z)\} \subseteq \text{sol}(E'),$$

preserving the soundness of the solution set under the updated equivalence partition E' .

3. **Algebraic equality:** For a constraint of the form $x = y + z$ with $d(z) = [0, 0]$, the semantics enforce

$$\text{asn}(x) = \text{asn}(y) + 0 = \text{asn}(y).$$

Therefore, merging the equivalence classes $[x]_E$ and $[y]_E$ is sound, as it enforces the necessary equality in the solution space.

4. **Constraint removal:** If $A(c) = 1$, then the constraint c is logically implied by the current domains and equivalences (d, E) . Formally,

$$\text{sol}(d, C) = \text{sol}(d, C \setminus \{c\}),$$

so removing c does not exclude any solution, preserving soundness.

By these points, the invariant holds at iteration $i + 1$. As the propagator iterations converge to a fixpoint, the invariant at fixpoint implies the theorem statement. $\square$

7 Parallel Correctness

Definition 13 (Meet Operation). A meet operation on the product lattice component $\mathcal{L}$ is an update operation of the form:

$$\text{Meet}(\ell, \ell') : \text{update } \ell \leftarrow \ell \wedge \ell'.$$

This operation is realized component-wise as:

- Min/max updates on domain interval endpoints.
- Union-find merges on partitions (requires atomic compare-and-swap for thread-safe concurrent union-find).
- Bitwise ORs on activity flags.

Remark 1. The correctness of the parallel algorithm relies on the algebraic properties of lattice meet operations rather than heavy synchronization. Each component update leverages:

- **Monotonicity:** The meet operation strictly refines or preserves the lattice element, ensuring information never regresses.
- **Commutativity and Associativity:** Component-wise meets commute (order-independent) and associate freely. This means parallel threads can apply propagators in any order, and the final result remains identical to sequential execution.
- **Idempotence:** Repeated applications of the same meet have no additional effect, making redundant computations harmless.

These lattice properties eliminate the need for fine-grained locking or atomic operations on domain bounds and activity flags. Only the union-find data structure requires atomic operations (compare-and-swap) to ensure thread-safe merging of equivalence classes. This design significantly reduces synchronization overhead and contention.

Algorithm 8 Parallel Fixpoint

```

1: function PARALLELSIMPLIFY(TCN  $\langle d_0, C \rangle$ ,  $T$  threads)
2:    $\ell \leftarrow (d_0, \top_{\mathcal{E}}, \perp_{\mathcal{A}})$  ▷ Shared global state
3:   repeat
4:      $\ell_{\text{old}} \leftarrow \ell$ 
5:     parallel for  $c \in C$  do ▷ Each thread processes subset
6:        $\ell_{\text{local}} \leftarrow$  apply propagators to  $c$ 
7:        $\text{MEET}(\ell, \ell_{\text{local}})$  ▷ Lattice meet update
8:     end parallel
9:     barrier
10:  until  $\ell = \ell_{\text{old}}$ 
11:  return  $\ell$ 
12: end function

```

Theorem 10 (Parallel Correctness). The parallel algorithm computes the same fixpoint as the sequential algorithm.

Proof. The key observation is that the lattice meet $\wedge$ is associative, commutative, and idempotent.

Let $\ell_1^{(c)}, \dots, \ell_T^{(c)}$ be the local updates computed by the threads for constraints in iteration i .

Sequential:

$$\ell^{(i+1)} = \ell^{(i)} \wedge \ell_1 \wedge \dots \wedge \ell_T.$$

Parallel: Meets in any order produce:

$$\ell^{(i+1)} = \ell^{(i)} \wedge \ell_{\sigma(1)} \wedge \dots \wedge \ell_{\sigma(T)},$$

for any permutation σ .

By commutativity and associativity, the sequential and parallel results coincide.

Convergence: Since all updates are monotone and information-increasing, both sequential and parallel iterations reach the same fixpoint (by uniqueness of the least fixpoint). $\square$

Proposition 1 (Time Complexity). *Let $n = |X|$ be the number of variables, $m = |C|$ the number of constraints, B the size of the bound set, and I the number of fixpoint iterations.*

- **Sequential:** *The time complexity is*

$$O(I \cdot m \cdot \alpha(n)),$$

where $\alpha(n)$ is the inverse Ackermann function, a very slowly growing function arising from union-find operations.

- **Parallel (idealized):** *With T threads and perfect load balancing and negligible synchronization overhead, the complexity reduces roughly to*

$$O\left(I \cdot \frac{m}{T} \cdot \alpha(n)\right).$$

- **Typical:** *The number of iterations is bounded by the lattice height*

$$I \leq h(\mathcal{L}) \leq n \cdot |B| + n,$$

which is practically much smaller than n^2 .

Remark 2. *Actual parallel performance is influenced by factors such as:*

- *Overhead and contention of atomic operations (CAS).*
- *Load imbalance among threads.*
- *Cache coherence traffic and memory bandwidth limitations.*

Consequently, empirical speedup is typically sub-linear but can be significant for large m .

8 Conclusion

We have presented a formally verified constraint simplifier grounded in lattice theory, and proved its correctness. The framework is based on a finite-height bounded lattice structure that guarantees termination and soundness, and monotone propagators which ensure entropy decreases at each step. Our soundness proof shows that the solution set is preserved throughout the simplification process. Furthermore, we introduced simple, parallelizable meet operations that enable correct coherent concurrent fixpoint computations.

8.1 Future Work

Several promising directions could extend this work:

Abstract Domain Refinement via Galois Connections. Galois connections offer a framework to lift computations from concrete domains (sets of integer intervals) to abstract domains (positive or negative interval) while preserving soundness. By establishing abstraction (α) and concretization (γ) functions satisfying $\alpha \circ \gamma \sqsubseteq \text{id}$ and $\text{id} \sqsubseteq \gamma \circ \alpha$, we can systematically design coarser abstract domains that may be used for initial, cheaper approximations before refining to the concrete domain. This could lead to more efficient constraint simplification by reducing the search space, while still guaranteeing soundness through the Galois connection properties.

Hypergraph-Based Parallel Decomposition. Representing the constraint network as a hypergraph, where variables are nodes and constraints are hyperedges connecting their scopes enables spatial locality optimizations. Graph partitioning algorithms can decompose the hypergraph into balanced subgraphs with minimal edge cuts, allowing threads to work on largely independent constraint subsets with reduced races. This approach could lower the amount of iterations needed for fixpoint convergence.